

Programação Funcional

Aula 04 - Char e Bool

Eliseu C. Miguel

Universidade Federal de Alfenas

11 de agosto de 2025

Tópicos

- 1 O Tipo Bool
- 2 O Tipo Char
- 3 Exercícios
- 4 Referências

Tópicos

- 1 O Tipo Bool
- 2 O Tipo Char
- 3 Exercícios
- 4 Referências

Tópicos

- 1 O Tipo Bool
- 2 O Tipo Char
- 3 Exercícios
- 4 Referências

Tópicos

- 1 O Tipo Bool
- 2 O Tipo Char
- 3 Exercícios
- 4 Referências

Booleanos em Haskell

- O tipo `Bool` representa valores lógicos.
- Possui apenas dois valores possíveis: `True` (verdadeiro) e `False` (falso).
- É fundamental para o controle de fluxo e lógica condicional.

Exemplos de Tipagem

```
True :: Bool
False :: Bool
3 > 2 :: Bool    (resulta em True)
```

Operadores Lógicos

- Haskell fornece os operadores lógicos padrão para trabalhar com Bool.

E Lógico (&&)

```
True && True = True  
True && False = False  
False && x = False
```

OU Lógico (||)

```
True || x = True  
False || True = True  
False || False = False
```

Negação (not)

```
not True = False  
not False = True
```

Funções com Bool

- Podemos usar casamento de padrão para definir funções com Bool.

Exemplo: Função "e exclusivo" (XOR)

```
xor :: Bool -> Bool -> Bool  
xor True False = True  
xor False True = True  
xor _ _ = False
```

Exemplo de execução

```
xor True True = False  
xor False True = True
```

Melhores Práticas com Booleanos

- Lembre-se que operadores de comparação (como $>$, $<$, $==$) já são funções que retornam Bool.
- Portanto, é redundante checar o resultado de uma comparação para então retornar True ou False.

Implementação Redundante

```
aMaiorQueB :: Int -> Int -> Bool
aMaiorQueB a b
  | a > b = True
  | otherwise = False
```

Melhor Prática

```
aMaiorQueB :: Int -> Int -> Bool
aMaiorQueB a b = a > b
```

Caracteres em Haskell

- O tipo Char representa um único caractere Unicode.
- Literais do tipo Char são escritos entre aspas simples.

Exemplos

```
'a' :: Char  
'Z' :: Char  
'7' :: Char
```

String é uma lista de Char

```
"Ababa" :: [Char]  
['A', 'b', 'a', 'b', 'a']  
(As duas formas são  
equivalentes)
```

Funções Úteis do Módulo Data.Char

- O módulo `Data.Char` contém muitas funções úteis para manipulação de caracteres.
- Para usá-lo em um arquivo, inclua `import Data.Char` no início.

Funções de Verificação (retornam Bool)

```
isUpper 'A' = True  
isLower 'b' = True  
isDigit '3' = True  
ord 'a' = 97  
chr 97 = 'a'
```

Exercícios

- 1 Escreva uma função que separa `[(Int,Char)]` em `([Int],[Char])`
- 2 Escreva uma função que recebe `[Char]` e retorna `[(Bool,Char)]`
True se Char for alfanumérico e False, caso contrário
- 3 Escreva uma função que recebe `[(Bool, Char)]` e filtra alfanuméricos
- 4 Escreva uma função que transforma `String ([Char])` de alfanuméricos em `Int`
- 5 Escreva uma função que gere a tabela ASCII

Referências Bibliográficas

Book [1]



Vladimir Oliveira Di Iorio.

Programação Funcional - Tipos Básicos e Programas Simples em Haskell.

DPI - UFV.

Sobre as apresentações didáticas

Atenção: este material não é completo para seus estudos.

Ao contrário, é apenas um guia para lhe informar quais são os tópicos importantes a serem estudados.

O estudo completo e necessário dá-se por materiais específicos sobre cada tópico, como livros e conteúdo na Internet.

Selecione materiais de fontes de seu interesse considerando a bibliografia citada no plano de ensino da disciplina para estudar os tópicos aqui abordados.

Consulte o professor, em caso de dúvidas

Agradecimentos especiais

Agradeço ao monitor da disciplina Felipe Araújo Correia por ter elaborado esta apresentação.

Agradeço a toda a comunidade $\text{\LaTeX}$.
Em especial a *Till Tantau* pelo *Beamer*.

<https://www.tcs.uni-luebeck.de/mitarbeiter/tantau/>

Desta forma, tornou-se possível a escrita deste material didático.